

Reviewer Pack — Minimal Number of Monomial Representations and Frege Proof D...

Entry 30492d1e2f0a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 22:21:36 UTC

Minimal Number of Monomial Representations and Frege Proof Depth Inequality

Entry ID: 30492d1e2f0a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 22:21:36 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Monomial Theory) **Field B** (complexity object): Boolean Satisfiability (Frege Proof Complexity)

Statement:

For every CNF formula φ , the minimal number of monomials required to represent the clause indicators in its Frege proof is at least logarithmic in the Frege proof depth of φ .

Rationale (proposer's reasoning):

Monomial theory provides a way to encode logical formulas algebraically. If the representation size is logarithmic in the proof depth, it suggests that the encoding captures essential complexity features of the formula's proof structure.

Taxonomy category: MONOMIAL_THEORY_TO_FREGE_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8182274e308bb5bf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the size of the minimal monomial representation and the logarithm of the Frege proof depth exceeds 0.5 for at least 80% of the seeds, with all seed metrics exceeding a threshold of 1. The conjecture is falsified if any seed produces a metric less than or equal to 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "monomial representation" AND "Frege proof complexity" - "Algebraic geometry" AND "Boolean satisfiability" AND "proof depth" - "CNF formula" AND "algebraic geometry" AND "Frege proof"

Top relevant hits considered: - [s2:f00d943cea727962981a17afc4650e9510d2008f] How a Math Class Can Be Two Places at Once: A Proposed Mathematical/Philosophical Collaboration

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n - 1):
            clause = [random.choice([1, -1]) * i for i in range(1, n+1)]
            clauses.append(clause)
        return clauses

    def frege_proof_depth(cnf):
        depth = 0
        stack = []
        for clause in cnf:
            if all(x > 0 for x in clause):
                depth += 1
                stack.append(depth)
            else:
                depth -= 1
                stack.pop()
        return max(stack) if stack else 0

    def monomial_representation(cnf):
        n = len(cnf[0])
        monomials = []
        for i in range(2**n):
            monomial = [int(x) for x in format(i, f'0{n}b')]
            if all(monomial[j] * cnf[j][i//2**j] >= 0 for j in range(n)):
                monomials.append(monomial)
```

```

        monomials.append(monomial)
    return monomials

def pearson_correlation(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
    var_x = sum((x[i] - mean_x)**2 for i in range(n)) / n
    var_y = sum((y[i] - mean_y)**2 for i in range(n)) / n
    return cov_xy / (math.sqrt(var_x) * math.sqrt(var_y))

results = []
for n in [5, 10, 15, 20, 30, 40]:
    cnf = generate_cnf(n)
    depth = frege_proof_depth(cnf)
    if depth == 0:
        continue
    monomials = monomial_representation(cnf)
    results.append((len(monomials), math.log(depth)))

if not results:
    return {
        "metric_name": "Pearson Correlation",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "No valid instances found"
    }

metric_value = pearson_correlation([x for x, _ in results], [y for _, y in results])
conjecture_holds = metric_value > 0.5
counterexample = "" if conjecture_holds else f"Metric value: {metric_value}"

return {
    "metric_name": "Pearson Correlation",
    "metric_value": metric_value,
    "instances_tested": len(results),
    "n_max": max(n for n, _ in results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2*i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)

```

```

support_fraction = 1.0
print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
elif any(result["metric_value"] <= 0.5 for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["metric_value"] <= 0.5)
    print(f"RESULT: FALSIFIED counterexample='Pearson correlation ≤ 0.5' first_failing_seed={first_failing_seed}")
else:
    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)
    print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f00be63b.py", line 96, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f00be63b.py", line 61, in run_trial
    depth = frege_proof_depth(cnf)
            ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f00be63b.py", line 37, in frege_proof_depth
    stack.pop()
IndexError: pop from empty list

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete the computation necessary to evaluate the conjecture. | next: Investigate and fix the crash in the test code to allow for a proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13910
2	preregistration	ollama_remote	glm4:latest	0	0	9340
3	novelty	ollama_remote	glm4:latest	0	0	8670
4	novelty	ollama_remote	glm4:latest	0	0	8981
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14710

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14506
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9573
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11433
9	judge	ollama_remote	glm4:latest	0	0	12181

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 103304 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/30492d1e2f0a.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/30492d1e2f0a.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/30492d1e2f0a.tar.gz (if generated)